

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1



BÁO CÁO ĐỀ TÀI CUỐI KÌ HỌC PHẦN
CHUYÊN ĐỀ HỆ THỐNG THÔNG TIN

Đề tài: Hệ thống phân loại email tự động

Giảng viên hướng dẫn: PGS.TS Phạm Văn Cường

Nhóm học phần: Nhóm 13

Sinh viên

Nguyễn Quốc Dương

Lâm Quốc Tú

Nguyễn Sơn Tùng

Mã sinh viên

B22DCCN167

B22DCCN743

B22DCCN768

Hà Nội, tháng 5 năm 2026

Lời nói đầu

Lời đầu tiên, nhóm chúng em xin được gửi lời cảm ơn chân thành và sâu sắc nhất tới các thầy cô giáo của Học viện đã tận tình giảng dạy, truyền đạt những kiến thức và kinh nghiệm quý báu trong suốt quá trình học tập. Những kiến thức đó là nền tảng quan trọng giúp chúng em thực hiện bài tập lớn này.

Nhóm chúng em xin gửi lời cảm ơn sâu sắc đến thầy Phạm Văn Cường - giảng viên học phần Chuyên đề Hệ thống thông tin, đã tận tình hướng dẫn, chỉ bảo và hỗ trợ chúng em trong suốt quá trình thực hiện bài tập lớn. Những góp ý và định hướng của thầy là cơ sở quan trọng giúp nhóm hoàn thành bài tập này.

Với những kiến thức được học cùng với sự hướng dẫn của thầy, nhóm chúng em đã cố gắng hoàn thành bài tập lớn này. Tuy nhiên, do không thể tránh khỏi những hạn chế về thời gian và kinh nghiệm, báo cáo khó tránh khỏi những thiếu sót. Nhóm rất mong nhận được sự góp ý từ thầy để có thể hoàn thiện hơn.

Trân trọng cảm ơn!

Mục lục

Danh sách bảng	4
Danh sách hình ảnh	5
CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI.....	6
1.1 Bối cảnh nghiên cứu	6
1.2 Lý do chọn đề tài.....	6
1.3 Mục tiêu nghiên cứu	7
1.4 Phạm vi nghiên cứu.....	7
1.5 Cấu trúc báo cáo.....	8
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1. Lý thuyết SVM	9
2.1.1. Hàm mục tiêu của SVM.....	9
2.1.2. Ràng buộc hàm mục tiêu.....	9
2.1.3. Bài toán tối ưu.....	9
2.1.4. Phương trình Lagrange và điều kiện KKT.....	9
2.1.5. Ước lượng λ bằng Gradient Descent.....	10
2.1.6. Tìm w và b	10
2.1.7. Tóm tắt thuật toán Gradient Descent cho SVM.....	11
2.2. Thuật toán Pegasos	11
2.2.1. Thuật toán Pegasos cơ bản.....	11
2.2.2. Hàm mục tiêu và hàm mất mát Hinge	11
2.2.3. Bước chiếu (Projection Step)	11
CHƯƠNG 3: PHƯƠNG PHÁP NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH	12
3.1 Tổng quan phương pháp	12
3.2 Mô tả dữ liệu	12
3.2.1 Nguồn dữ liệu.....	12
3.2.2 Tạo tập mẫu cho thực nghiệm.....	12
3.2.3 Chia tập dữ liệu	13
3.3 Tiền xử lý dữ liệu.....	13
3.3.1 Loại bỏ ký tự đặc biệt và số	13
3.3.2 Loại bỏ stopwords.....	14

3.3.3 Loại bỏ URL	14
3.3.4 Tách từ	15
3.3.5 Stemming	15
3.4 Vector hoá đặc trưng bằng TF-IDF	17
3.5 Xây dựng mô hình phân loại.....	18
3.5.1 Mô hình Hard-margin SVM.....	18
3.5.2 Mô hình SVM với thuật toán Pegasos SGD	20
3.6 Huấn luyện mô hình.....	21
3.6.1 Quy trình huấn luyện	21
3.6.2 Tiêu chí đánh giá.....	21
CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM.....	23
4.1. Tổng quan thực nghiệm	23
4.2. Kết quả so sánh các mô hình.....	23
4.3. Ảnh hưởng của kích thước tập huấn luyện	24
4.3.1. Độ chính xác và F1-Score.....	24
4.3.2. Thời gian huấn luyện	26
4.4. So sánh với mô hình học máy khác	27
4.5. Kết quả triển khai hệ thống	27
4.6. Nhận xét và đánh giá.....	29
CHƯƠNG 5: KẾT LUẬN	31

Danh sách bảng

Bảng 4.1: Kết quả đánh giá tổng thể các mô hình SVM	23
Bảng 4.2: Kết quả SVC và SVM Pegasos theo kích thước huấn luyện.....	24
Bảng 4.3: Đánh giá so sánh SVM và các mô hình khác	27

Danh sách hình ảnh

Ảnh 3.1: file create_sample_dataset.py	13
Ảnh 3.2: Script loại bỏ ký tự đặc biệt và số.....	14
Ảnh 3.3: Script loại bỏ stopwords.....	14
Ảnh 3.4: Script loại bỏ URL	15
Ảnh 3.5: Script tách từ	15
Ảnh 3.6: Các từ sau khi được rút gọn	16
Ảnh 3.7: Script Stemming.....	16
Hình 3.8: Công thức tính TF và IDF.....	17
Hình 3.9: Script áp dụng TF-IDF vectorization.....	17
Hình 3.10: Cài đặt HardMarginSVM.....	19
Hình 3.11: Cài đặt SVM với thuật toán Pegasos	20
Hình 4.1: Phân phối nhãn trong tập dữ liệu mẫu (ham: label 0, spam: label 1)	23
Hình 4.2: Accuracy của ba mô hình theo kích thước tập huấn luyện.....	25
Hình 4.4: Thời gian huấn luyện (ms) của ba mô hình theo kích thước tập huấn luyện	27
Hình 4.7: Giao diện hệ thống phân loại email - kết quả: không phải SPAM (ham).28	
Hình 4.8: Giao diện hệ thống phân loại email - kết quả: có khả năng là SPAM.....	29

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh nghiên cứu

Trong thời đại số hóa hiện nay, thư điện tử (email) đã trở thành một trong những phương tiện giao tiếp thiết yếu và phổ biến nhất trong môi trường làm việc và cuộc sống cá nhân trên toàn thế giới. Với hàng tỷ email được gửi đi mỗi ngày, hệ thống thư điện tử đóng vai trò không thể thiếu trong việc trao đổi thông tin, phối hợp công việc và duy trì các mối quan hệ xã hội.

Tuy nhiên, sự phát triển mạnh mẽ của hệ thống email cũng kéo theo một vấn nạn nghiêm trọng: thư rác (spam email). Theo báo cáo của Kaspersky, 44.99% tổng lưu lượng email lưu thông trên Internet năm 2025 là spam. Đây là những email không mong muốn được gửi hàng loạt với các mục đích như:

- Quảng cáo sản phẩm, dịch vụ không có yêu cầu từ người nhận.
- Lừa đảo tài chính, đánh cắp thông tin cá nhân và tài khoản.
- Phát tán mã độc, phần mềm gián điệp và các mối đe dọa an ninh mạng.

Sự tồn tại của spam email không chỉ gây lãng phí tài nguyên hệ thống và băng thông mạng, mà còn tiềm ẩn rủi ro bảo mật nghiêm trọng đối với cá nhân, tổ chức và doanh nghiệp. Việc xử lý thủ công hàng chục, hàng trăm thư rác mỗi ngày gây tốn kém thời gian và làm giảm năng suất làm việc đáng kể.

Trước thực trạng đó, các hệ thống lọc spam tự động dựa trên học máy (Machine Learning) đã được nghiên cứu và ứng dụng rộng rãi. Trong số các thuật toán học máy được áp dụng cho bài toán phân loại văn bản, Support Vector Machine (SVM) nổi bật như một phương pháp hiệu quả, có nền tảng lý thuyết vững chắc và khả năng tổng quát hóa tốt, đặc biệt phù hợp với dữ liệu văn bản có chiều cao như email.

1.2 Lý do chọn đề tài

Mặc dù đã có nhiều nghiên cứu về phân loại spam email, hầu hết các hệ thống thực tế hiện nay chỉ tập trung vào sử dụng các thư viện học máy có sẵn mà không đi sâu vào bản chất và cơ chế hoạt động bên trong của thuật toán. Điều này tạo ra khoảng cách lớn giữa lý thuyết và thực tiễn, đặc biệt đối với những người học muốn hiểu và ứng dụng SVM một cách toàn diện.

Đề tài "Hệ thống phân loại email tự động sử dụng Support Vector Machine" được lựa chọn xuất phát từ các lý do cụ thể sau:

- Tính thực tiễn cao: Spam email là vấn đề phổ biến, ảnh hưởng trực tiếp đến người dùng hàng ngày, tạo động lực nghiên cứu rõ ràng và có thể đánh giá kết quả một cách cụ thể.
- Giá trị học thuật: SVM là một trong những thuật toán học máy cổ điển nhưng có nền tảng toán học sâu sắc. Việc tự cài đặt SVM giúp hiểu rõ bản chất tối ưu hóa và cơ chế phân loại.

- Chuỗi xử lý đầy đủ: Đề tài bao phủ toàn bộ pipeline từ thu thập dữ liệu, tiền xử lý văn bản (NLTK), vector hóa (TF-IDF), huấn luyện mô hình đến triển khai ứng dụng demo thực tế (Streamlit).
- Khả năng so sánh và đánh giá: Thông qua việc triển khai và thực nghiệm với SVM trên các tập dữ liệu có kích thước khác nhau, đề tài cho phép phân tích toàn diện về hiệu năng và khả năng mở rộng của từng phương pháp.

1.3 Mục tiêu nghiên cứu

Đề tài hướng đến các mục tiêu chính sau:

- Nghiên cứu lý thuyết về Support Vector Machine (SVM), bao gồm bài toán tối ưu hóa, hyperplane.
- Tự cài đặt thuật toán SVM từ đầu theo phương pháp Pegasos Stochastic Gradient Descent (SGD) và Hard-margin SVM, không phụ thuộc hoàn toàn vào thư viện scikit-learn.
- Xây dựng quy trình tiền xử lý văn bản email tiếng Anh sử dụng thư viện NLTK: loại ký tự đặc biệt, loại URL, tách từ (tokenization), loại stopwords và chuẩn hóa từ (stemming).
- Ứng dụng TF-IDF (Term Frequency - Inverse Document Frequency) để vector hóa văn bản, biểu diễn email dưới dạng đặc trưng số phục vụ huấn luyện mô hình.
- Thực nghiệm và so sánh hiệu năng của SVM trên các tập dữ liệu có kích thước khác nhau, đánh giá theo các chỉ số Accuracy, Precision, Recall và F1-score.
- Triển khai hệ thống phân loại email dưới dạng ứng dụng web demo sử dụng Streamlit, cho phép người dùng nhập nội dung email và nhận kết quả phân loại spam/non-spam theo thời gian thực.

1.4 Phạm vi nghiên cứu

Trong khuôn học phần Chuyên đề hệ thống thông tin, đề tài tập trung vào:

- Phân loại email viết bằng tiếng Anh.
- Bài toán phân loại nhị phân: spam (thư rác) và non-spam (thư thường).
- Đặc trưng đầu vào dựa trên nội dung văn bản, được vector hóa bằng TF-IDF.
- Các phương pháp học máy: SVM tuyến tính (cài đặt thủ công theo Pegasos SGD và Hard-margin), và SVM tuyến tính sử dụng scikit-learn.
- Triển khai ứng dụng demo với Streamlit, hướng đến khả năng sử dụng thực tế.

Đề tài không đi sâu vào:

- Phân tích email đa ngôn ngữ ngoài tiếng Anh.
- Các mô hình học sâu như RNN, LSTM, hay Transformer.

- Phân tích siêu dữ liệu email, như địa chỉ IP nguồn, thông tin header mạng.
- Xử lý nội dung đính kèm phi văn bản (hình ảnh, tệp PDF, v.v.).

1.5 Cấu trúc báo cáo

Báo cáo được tổ chức thành các chương sau:

- Chương 1: Giới thiệu đề tài - Trình bày bối cảnh nghiên cứu, lý do chọn đề tài, mục tiêu, phạm vi và cấu trúc của báo cáo.
- Chương 2: Cơ sở lý thuyết - Giới thiệu các kiến thức nền tảng liên quan đến bài toán phân loại văn bản, lý thuyết SVM (Hard-margin, Soft-margin, Pegasos), phương pháp tiền xử lý văn bản và vector hóa TF-IDF.
- Chương 3: Phương pháp nghiên cứu và xây dựng mô hình - Mô tả chi tiết quy trình xây dựng hệ thống, bao gồm thu thập và chuẩn bị dữ liệu, tiền xử lý văn bản, cài đặt SVM, và kiến trúc ứng dụng Streamlit.
- Chương 4: Kết quả thực nghiệm - Trình bày môi trường thực nghiệm, tập dữ liệu sử dụng, quy trình huấn luyện và các kết quả đo lường hiệu năng của các mô hình theo các kích thước dữ liệu khác nhau.
- Chương 5: Đánh giá mô hình - Phân tích và so sánh kết quả thực nghiệm của mô hình, đánh giá ưu nhược điểm, rút ra kết luận và đề xuất hướng phát triển tiếp theo.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Lý thuyết SVM

2.1.1. Hàm mục tiêu của SVM

Mục tiêu của SVM là tối ưu hóa khoảng cách giữa các điểm dữ liệu của hai lớp. Hàm mục tiêu được xây dựng sao cho margin giữa hai lớp là lớn nhất, đồng thời đảm bảo rằng không có điểm nào bị sai phân loại.

Nếu dữ liệu huấn luyện có thể phân tách tuyến tính, có thể chọn hai siêu phẳng song song để phân tách hai lớp dữ liệu, sao cho khoảng cách giữa chúng là lớn nhất có thể. Khu vực được giới hạn bởi hai siêu phẳng này được gọi là "margin" (biên). Siêu phẳng có margin lớn nhất là siêu phẳng nằm ở giữa hai siêu phẳng này.

Với một bộ dữ liệu đã chuẩn hóa, các siêu phẳng này có thể được mô tả bằng các phương trình:

- $w^T x + b = 1$ (mọi điểm trên hoặc phía trên ranh giới này thuộc về lớp có nhãn 1)
- $w^T x + b = -1$ (mọi điểm trên hoặc phía dưới ranh giới này thuộc về lớp có nhãn -1)

Về mặt hình học, khoảng cách giữa 2 siêu phẳng là $2/\|w\|$, nên ta cần tối thiểu hóa $\|w\|$.

2.1.2. Ràng buộc hàm mục tiêu

Cần ngăn không cho các điểm dữ liệu rơi vào margin. Với mỗi điểm dữ liệu thứ i , ràng buộc là:

- Nếu $y_i = 1$: $w^T x_i + b \geq 1$
- Nếu $y_i = -1$: $w^T x_i + b \leq -1$

Có thể viết gọn lại thành: $y_i(w^T x_i + b) \geq 1, \forall 1 \leq i \leq n$

2.1.3. Bài toán tối ưu

Tóm lại, ta cần giải bài toán tối ưu:

$$\min_{w,b} \frac{\|w\|^2}{2}$$

với ràng buộc: $y_i(w^T x_i + b) \geq 1, \forall i \in \{1, \dots, n\}$. Cần tìm w và b .

2.1.4. Phương trình Lagrange và điều kiện KKT

2.1.4.1. Hàm Lagrange

Hàm Lagrange của bài toán được định nghĩa như sau:

$$\mathcal{L}(w, b, \lambda) = \frac{1}{2}\|w\|^2 - \sum_i \lambda_i [y_i(w^T x_i + b) - 1]$$

trong đó $\lambda_i \geq 0$ là các hệ số Lagrange.

2.1.4.2. Điều kiện KKT

Ba điều kiện KKT cần thỏa mãn:

- $\lambda_i \geq 0, \forall i = 1, \dots, n$
- $y^{(i)}(w \cdot x^{(i)} + b) - 1 \geq 0, \forall i = 1, \dots, n$
- $\lambda_i [y^{(i)}(w \cdot x^{(i)} + b) - 1] = 0, \forall i = 1, \dots, n$

Từ điều kiện thứ ba suy ra: với mỗi i , $\lambda_i = 0$ hoặc $y^{(i)}(w \cdot x^{(i)} + b) = 1$. Các điểm thỏa mãn $\lambda_i > 0$ nằm trên margin được gọi là các **vector hỗ trợ (support vectors)**.

2.1.4.3. Rút gọn hàm Lagrangian

Từ đạo hàm của $\mathcal{L}$ theo w và b ta có:

$$w = \sum_{i=1}^n \lambda_i y^{(i)} x^{(i)}$$

Và

$$\sum_{i=1}^n \lambda_i y^{(i)} = 0$$

Thay vào hàm Lagrangian, ta được dạng dual:

$$L(w, b, \lambda_1, \dots, \lambda_n) = \sum_{i=1}^n \lambda_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j y^{(i)} y^{(j)} x^{(i)T} x^{(j)} = \tilde{L}(\lambda_1, \dots, \lambda_n)$$

2.1.5. Ước lượng λ bằng Gradient Descent

Bài toán quy về tối đa hóa $\tilde{L}(\lambda)$ với ràng buộc $\lambda_i \geq 0$ và $\sum_i \lambda_i y^{(i)} = 0$. Phương pháp Gradient Descent được áp dụng với quy tắc cập nhật:

$$\lambda^{[t+1]} = \lambda^{[t]} + \eta \cdot \frac{\partial \tilde{L}(\lambda)}{\partial \lambda} = \lambda^{[t]} + \eta \cdot (1 - H \lambda^{[t]})$$

trong đó η là tốc độ học (learning rate), H là ma trận $n \times n$ với phần tử $(H)_{ij} = y^{(i)} y^{(j)} x^{(i)T} x^{(j)}$ (ma trận Hadamard giữa yy^T và XX^T). Cần đảm bảo ràng buộc $\lambda_i \geq 0$ bằng cách cắt giá trị âm về 0 sau mỗi bước cập nhật.

2.1.6. Tìm $\hat{w}$ và $\hat{b}$

Sau khi ước lượng được $\hat{\lambda}$, vector trọng số $\hat{w}$ được tính từ các support vectors:

$$\hat{w} = \sum_{i \in \text{Support Vector}}^n \hat{\lambda}_i y^{(i)} x^{(i)}$$

trong đó S là tập hợp các support vectors (điểm có $\hat{\lambda}_i > 0$). Tham số $\hat{b}$ được tính trung bình trên tất cả support vectors để giảm sai số:

$$\hat{b} = \frac{1}{|S|} \sum_{x^{(i)} \in S} y^{(i)} - \hat{w}^T x^{(i)}$$

2.1.7. Tóm tắt thuật toán Gradient Descent cho SVM

1. Khởi tạo: $\lambda[0]$ ngẫu nhiên, tính ma trận H .
2. Lặp: Cập nhật λ theo công thức $\lambda[t+1] = \lambda[t] + \eta(1 - H\lambda[t])$.
3. Áp dụng ràng buộc: $\lambda_i \leftarrow \max(0, \lambda_i)$.
4. Tìm support vectors: $S = \{i : \lambda_i > 0\}$.
5. Tính tham số mô hình: $\hat{w}$ và $\hat{b}$ từ các công thức trên.

2.2. Thuật toán Pegasos

2.2.1. Thuật toán Pegasos cơ bản

Pegasos (Primal Estimated sub-GrAdient SOLver for SVM) là thuật toán tối ưu hóa hiệu quả cho SVM trên tập dữ liệu lớn. Thuật toán hoạt động trên primal form với mini-batch stochastic gradient descent.

2.2.2. Hàm mục tiêu và hàm mất mát Hinge

Cần tìm vector trọng số w để tối thiểu hóa hàm mục tiêu:

$$f(w, X, Y) = \frac{\lambda}{2} \cdot \|w\|^2 + \frac{1}{|Y|} \cdot \sum_i \text{Loss}(w, x_i, y_i)$$

Hàm mất mát Hinge được sử dụng cho SVM:

$$\text{Loss}(w, x_i, y_i) = \max(0, 1 - y_i \cdot (w \cdot x_i))$$

Dạng tường minh của hàm mất mát Hinge:

- $1 - y_i \cdot (w \cdot x_i)$ nếu $y_i \cdot (w \cdot x_i) < 1$
- 0 nếu $y_i \cdot (w \cdot x_i) \geq 1$

Pegasos tối ưu hóa hàm mục tiêu theo từng mini-batch A_t với k mẫu:

$$f(w; A_t) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{k} \sum_{i \in A_t} \ell(w, x_i, y_i)$$

2.2.3. Bước chiếu (Projection Step)

Để giới hạn tập nghiệm khả thi trong phạm vi $\frac{1}{\sqrt{\lambda}}$, sau mỗi vòng lặp, w_t được chiếu lại:

$$w_{t+1} = w_{t+1} \cdot \min\left(1, \frac{1}{\sqrt{\lambda} \|w_{t+1}\|}\right)$$

CHƯƠNG 3: PHƯƠNG PHÁP NGHIÊN CỨU VÀ XÂY DỰNG MÔ HÌNH

3.1 Tổng quan phương pháp

Nhóm xây dựng hệ thống phân loại email spam sử dụng thuật toán Support Vector Machine. Phương pháp được triển khai theo hướng học máy có giám sát, trong đó mô hình SVM được cài đặt thủ công theo hai biến thể chính: Hard-margin SVM và SVM tối ưu hóa bởi thuật toán Pegasos SGD, kết hợp với kỹ thuật biểu diễn văn bản TF-IDF cho bài toán phân loại nhị phân.

Pipeline tổng thể của hệ thống bao gồm sáu bước chính được thực hiện tuần tự: Thu thập và chuẩn bị dữ liệu → Tiền xử lý văn bản → Vector hóa đặc trưng bằng TF-IDF → Xây dựng và cài đặt mô hình phân loại, huấn luyện và đánh giá mô hình → Triển khai ứng dụng demo.

3.2 Mô tả dữ liệu

3.2.1 Nguồn dữ liệu

Dữ liệu sử dụng trong nghiên cứu là tập hợp các email tiếng Anh được gán nhãn, có tên đầy đủ là "Combined Spam Email CSV of 2007 TREC Public Spam Corpus and Enron-Spam Dataset", được thu thập và công bố công khai trên nền tảng Kaggle.

Tổng cộng bộ dữ liệu bao gồm 83.448 bản ghi email, được phân loại thành hai nhãn là spam và non-spam. Trong đó, số lượng email spam là 43.910 mẫu và số lượng email non-spam là 39.538 mẫu. Tỷ lệ phân bố giữa hai lớp tương đối cân bằng: khoảng 52,6% spam và 47,4% non-spam.

Mỗi mẫu dữ liệu trong tập gốc bao gồm hai trường thông tin chính. Trường *label* chứa nhãn phân loại nhị phân, với giá trị 1 tương ứng với email spam và giá trị 0 tương ứng với email non-spam. Trường *text* chứa nội dung văn bản nguyên thủy của email, thường là chuỗi nhiều dòng, có thể bao gồm các thẻ HTML, URL, ký tự đặc biệt và nhiều loại nội dung hỗn tạp khác.

3.2.2 Tạo tập mẫu cho thực nghiệm

Do kích thước bộ dữ liệu gốc khá lớn, và một số biến thể thuật toán SVM như Hard-margin có độ phức tạp tính toán bậc hai theo số lượng mẫu, nhóm đã tạo file *create_sample_dataset.py* để tạo các tập mẫu có kích thước khác nhau phục vụ mục đích thực nghiệm và so sánh.

```
create_sample_dataset.py X
create_sample_dataset.py > ...
1
2
3 import pandas as pd
4
5 df = pd.read_csv("https://media.githubusercontent.com/media/PTIT-Assignment-Pro
6 print(df.shape)
7
8 target_column = 'label' # Replace with the actual name of your target column
9 proportions = df[target_column].value_counts(normalize=True)
10
11 # Calculate the number of samples for each class
12 a = [i for i in range(1000, 82000, 5000)]
13 for i in a:
14     num_samples = i
15     sample_counts = (proportions * num_samples).round().astype(int)
16
17     # Sample the dataset
18     sampled_df = pd.concat([
19         df[df[target_column] == label].sample(n=count, random_state=42)
20         for label, count in sample_counts.items()
21     ])
22
23     # Reset index if needed
24     sampled_df.reset_index(drop=True, inplace=True)
25     sampled_df.to_csv(f"dataset/sampled_dataset{i}.csv", index=False)
26
```

Ảnh 3.1: file *create_sample_dataset.py*

Script hoạt động bằng cách đọc file dữ liệu gốc → Tính toán tỉ lệ phân bố của từng lớp nhãn, sau đó thực hiện lấy mẫu phân tầng (stratified sampling) với *random_state=42* để đảm bảo tính tái lập của thực nghiệm. Kết quả là các file *sampled_dataset.csv* với N chạy từ 1.000 đến hơn 80.000 theo bước nhảy 5.000 mẫu.

3.2.3 Chia tập dữ liệu

Với mỗi tập mẫu, dữ liệu được chia thành tập huấn luyện (training set) và tập kiểm tra (test set) theo tỉ lệ 80/20 bằng hàm *train_test_split* của scikit-learn với *random_state=42*. Phân chia cố định này đảm bảo kết quả đánh giá khách quan và có thể so sánh nhất quán giữa các phương pháp và các lần thử nghiệm khác nhau.

3.3 Tiền xử lý dữ liệu

Tiền xử lý văn bản đóng vai trò quan trọng trong việc chuẩn hóa dữ liệu đầu vào và nâng cao chất lượng biểu diễn đặc trưng cho mô hình. Nội dung email thô thường chứa nhiều thành phần gây nhiễu như URL, ký tự đặc biệt, thẻ HTML và các từ không mang thông tin phân biệt. Quy trình tiền xử lý trong dự án bao gồm các bước được áp dụng nhất quán trong cả giai đoạn huấn luyện và triển khai.

3.3.1 Loại bỏ ký tự đặc biệt và số

Hàm *remove_special_characters()* được áp dụng để loại bỏ tất cả các ký tự không phải chữ cái và khoảng trắng. Bước này loại bỏ các ký hiệu, dấu câu, chữ số và ký tự đặc

biệt thường xuất hiện trong spam như "\$", "!", "%", hay các chuỗi số điện thoại và mã giảm giá.

```
In [10]: def remove_special_characters(word):
          return re.sub(r'^a-zA-Z\s', '', word)

In [11]: text = 'Hello everyone ! I am happy to meet 3 of us, my email is admin@gmail.com'
          print(remove_special_characters(text))

Hello everyone I am happy to meet  of us my email is admingmailcom
```

Ảnh 3.2: Script loại bỏ ký tự đặc biệt và số

3.3.2 Loại bỏ stopwords

Danh sách stopwords tiếng Anh từ `nlk.corpus.stopwords` được sử dụng để loại bỏ các từ xuất hiện quá phổ biến nhưng không mang thông tin phân biệt giữa hai lớp, như "the", "is", "are", "and", "a", "an". Hàm `remove_stop_words()` lọc ra những token không thuộc tập stopwords và trả về danh sách token đã lọc. Việc loại bỏ stopwords giúp giảm kích thước không gian đặc trưng và tập trung mô hình vào các từ mang ý nghĩa thực sự.

```
In [12]: ENGLISH_STOP_WORDS = set(stopwords.words('english'))

In [13]: def remove_stop_words(words):
          return [word for word in words if word not in ENGLISH_STOP_WORDS]

In [14]: print(' '.join(remove_stop_words(text.split())))

Hello everyone ! I happy meet 3 us, email admin@gmail.com
```

Ảnh 3.3: Script loại bỏ stopwords

3.3.3 Loại bỏ URL

Hàm `remove_url()` sử dụng biểu thức `http\S+` để phát hiện và thay thế tất cả các chuỗi bắt đầu bằng "http" bằng chuỗi rỗng. Đây là bước cần thiết vì URL thường chứa các tên miền và tham số ngẫu nhiên không mang giá trị phân biệt ngữ nghĩa, đồng thời làm tăng kích thước từ điển của vectorizer một cách không cần thiết.

Xoá các link website trong câu

```
In [15]: def remove_url(word):  
         return re.sub(r"http\S+", "", word)  
  
In [16]: text = 'My websites are https://google.com and https://reddit.com'  
         print(remove_url(text))  
  
My websites are  and
```

Ảnh 3.4: Script loại bỏ URL

3.3.4 Tách từ

Văn bản sau khi được làm sạch được đưa qua hàm `word_tokenize` của thư viện NLTK để tách thành danh sách các từ đơn lẻ. Bước tách từ tạo ra đơn vị cơ bản để thực hiện các bước xử lý tiếp theo như loại stopwords và stemming.

```
https://www.nltk.org/api/nltk.tokenize.word_tokenize.html https://www.geeksforgeeks.org/python-nltk-tokenizer-word_tokenize/  
https://www.geeksforgeeks.org/tokenize-text-using-nltk-python/  
  
df['text'] = df['text'].apply(remove_special_characters)  
df['text'] = df['text'].apply(remove_url)  
df['text'] = df['text'].apply(word_tokenize)  
df['text'] = df['text'].apply(remove_stop_words)  
df['text'] = df['text'].apply(' '.join)  
  
df.head()  
  
...  


|   | label | text                                              |
|---|-------|---------------------------------------------------|
| 0 | 1     | wireless optical button scroll mouse limited s... |
| 1 | 1     | desk philip moore dept credit control unit uni... |
| 2 | 1     | submitting website search engines may increase... |
| 3 | 1     | managers wanted inc escapenumber company looki... |
| 4 | 1     | anatrim extremely efficient flesh loss blend r... |


```

Ảnh 3.5: Script tách từ

3.3.5 Stemming

Bước cuối cùng trong quy trình tiền xử lý là chuẩn hóa các từ về dạng gốc bằng thuật toán stemming. Thư viện `SnowballStemmer('english')` của NLTK được sử dụng để áp dụng stemming cho từng token trong danh sách.

Ví dụ, các từ "running", "runs", "runner" đều được chuyển về dạng gốc "run", hay "university" được rút gọn về "univers".

Kết quả là danh sách token đã stem được nối lại thành một chuỗi văn bản bằng dấu cách. Stemming giúp giảm sự đa dạng hình thái học của từ vựng và đồng nhất các biến thể ngữ pháp của cùng một từ gốc, từ đó cải thiện chất lượng biểu diễn vector

Word	Stem
cared	care
university	univers
fairly	fair
easily	easili
singing	sing
sings	sing
sung	sung
singer	singer
sportingly	sport

Ảnh 3.6: Các từ sau khi được rút gọn

```
In [19]: stemmer = SnowballStemmer('english')

def stem_text(text):
    tokens = nltk.word_tokenize(text)

    stemmed_tokens = [stemmer.stem(token) for token in tokens]

    return ' '.join(stemmed_tokens)
```

```
In [20]: df['text'] = df['text'].apply(stem_text)
df.head()
```

Out[20]:

	label	text
0	1	wireless optic button scroll mous limit stock ...
1	1	desk philip moor dept credit control unit unio...
2	1	submit websit search engin may increas onlin s...
3	1	manag want inc escapenumb compani look manag g...
4	1	anatrism extrem effici flesh loss blend readili...

Ảnh 3.7: Script Stemming

3.4 Vector hoá đặc trưng bằng TF-IDF

Sau giai đoạn tiền xử lý, văn bản cần được chuyển đổi thành dạng số để mô hình học máy có thể xử lý. Nhóm sử dụng phương pháp TF-IDF (Term Frequency - Inverse Document Frequency) thông qua lớp `TfidfVectorizer` của `scikit-learn`.

TF-IDF là phương pháp biểu diễn văn bản theo dạng vector số, trong đó giá trị của mỗi chiều phản ánh tầm quan trọng của một từ trong một tài liệu cụ thể so với toàn bộ tập corpus.

Trọng số TF-IDF của một từ được tính theo công thức kết hợp giữa hai thành phần:

- Term Frequency (TF) đo tần suất xuất hiện của từ trong một tài liệu
- Inverse Document Frequency (IDF) đo mức độ hiếm của từ đó trong toàn bộ corpus.

$$TF = \frac{\text{Number of times a word "X" appears in a Document}}{\text{Number of words present in a Document}}$$

$$IDF = \log \left(\frac{\text{Number of Documents present in a Corpus}}{\text{Number of Documents where word "X" has appeared}} \right)$$

$$TF\ IDF = TF * IDF$$

Hình 3.8: Công thức tính TF và IDF

Những từ xuất hiện nhiều trong một email nhưng ít trong toàn bộ tập dữ liệu sẽ nhận trọng số cao, phản ánh tính đặc trưng của chúng cho tài liệu đó.

Trong hệ thống của nhóm, `TfidfVectorizer` được khởi tạo với các tham số mặc định và thực hiện `fit_transform` trên toàn bộ tập dữ liệu văn bản đã tiền xử lý. Kết quả thu được là ma trận đặc trưng dạng thưa (sparse matrix) với kích thước (số mẫu $\times$ số từ trong từ điển).

TF-IDF vectorization ¶

<https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/>

```
[21]: vectorizer = TfidfVectorizer()

      X = vectorizer.fit_transform(df['text'])
      print(X.shape)
```

(83448, 234926)

Hình 3.9: Script áp dụng TF-IDF vectorization

Khi áp dụng trên toàn bộ 83.448 mẫu của bộ dữ liệu gốc, ma trận TF-IDF có kích thước (83.448, 234.926), nghĩa là không gian đặc trưng lên tới hơn 234 nghìn chiều. Đây chính là lý do tại sao việc sử dụng SVM tuyến tính với biểu diễn TF-IDF sparse, vì SVM tuyến tính có khả năng xử lý tốt trong không gian đặc trưng chiều cao. Vectorizer sau khi được huấn luyện (fitted) được lưu thành file vectorizer.pkl

3.5 Xây dựng mô hình phân loại

3.5.1 Mô hình Hard-margin SVM

Biến thể đầu tiên được cài đặt là Hard-margin SVM, dựa trực tiếp trên lý thuyết tối ưu hóa bài toán dual thông qua nhân tử Lagrange. Lớp HardMarginSVM được triển khai với các tham số khởi tạo gồm tốc độ học $\eta=0.001$, số vòng lặp $\text{epoch}=1000$ và $\text{random_state}=42$.

Trong phương thức $\text{fit}(X, y)$, mô hình thực hiện chuyển đổi nhãn từ dạng nhị phân $\{0, 1\}$ sang dạng $\{-1, +1\}$ phù hợp với bài toán SVM. Vector trọng số w được khởi tạo bằng không và hệ số bias $b = 0$.

```

class HardMarginSVM:
    def __init__(self, eta=0.001, epoch=1000, random_state=42):
        self.eta = eta
        self.epoch = epoch
        self.random_state = random_state
        self.is_trained = False
        self.support_vectors = None

    def fit(self, X, y):
        if hasattr(X, "toarray"):
            X = X.toarray()
        self.num_samples = X.shape[0]
        self.num_features = X.shape[1]
        y_unique = np.unique(y)
        if len(y_unique) != 2:
            raise ValueError("Binary classification requires exactly 2 classes")
        if set(y_unique) == {0, 1}:
            y = np.where(y == 0, -1, 1)
        self.w = np.zeros(self.num_features)
        self.b = 0
        rgen = np.random.RandomState(self.random_state)
        self.alpha = rgen.uniform(low=0.0, high=0.01, size=self.num_samples)
        for i in range(self.epoch):
            self._cycle(X, y)
        sv_indices = np.where(self.alpha != 0)[0]
        self.support_vectors = sv_indices
        self.w = np.zeros(self.num_features)
        for i in sv_indices:
            self.w += self.alpha[i] * y[i] * X[i]
        bias_sum = 0
        for i in sv_indices:
            bias_sum += y[i] - np.dot(self.w, X[i])
        self.b = bias_sum / len(sv_indices)
        self.is_trained = True
        return self

    def predict(self, X):
        if not self.is_trained:
            raise Exception('Model not trained yet')
        if hasattr(X, "toarray"):
            X = X.toarray()
        decision_values = X @ self.w + self.b
        result = np.where(decision_values >= 0, 1, 0)
        return result

    def _cycle(self, X, y):
        y = y.reshape([-1, 1]) # (n, 1)
        XXT = X @ X.T
        H = (y @ y.T) * XXT
        grad = np.ones(self.num_samples) - H @ self.alpha
        self.alpha += self.eta * grad
        self.alpha = np.clip(self.alpha, 0, None)

```

Hình 3.10: Cài đặt HardMarginSVM

Tuy nhiên, Hard-margin SVM tồn tại một hạn chế nghiêm trọng về hiệu năng tính toán. Trong mỗi vòng lặp, thuật toán cần tính toán ma trận kernel H có kích thước $(n \times n)$ trong đó n là số lượng mẫu huấn luyện. Điều này dẫn đến độ phức tạp tính toán bậc hai $O(n^2)$, khiến thời gian huấn luyện tăng rất nhanh khi kích thước dữ liệu tăng lên.

3.5.2 Mô hình SVM với thuật toán Pegasos SGD

Để khắc phục hạn chế về hiệu năng của Hard-margin SVM, nhóm cài đặt thuật toán Pegasos (Primal Estimated sub-GrAdient SOLver for SVM) theo phương pháp Gradient Descent. Đây là phương pháp cho phép giải bài toán SVM trong không gian primal với độ phức tạp tính toán tuyến tính theo số lượng mẫu.

```
self.num_samples, self.num_features = X.shape
y_unique = np.unique(y)
if len(y_unique) != 2:
    raise ValueError("Phân loại nhị phân cần 2 nhãn")
if set(y_unique) == {0, 1}:
    y = np.where(y == 0, -1, 1)
self.w = np.zeros(self.num_features, dtype=np.float32)
self.b = 0.0
np.random.seed(self.random_state)
t = 0
previous_objective = float("inf")
for ep in range(1, self.epoch + 1):
    indices = np.random.permutation(self.num_samples)
    for start in range(0, self.num_samples, self.batch_size):
        t += 1
        end = start + self.batch_size
        batch_idx = indices[start:end]
        X_batch = X[batch_idx]
        y_batch = y[batch_idx]

        eta = 1.0 / (self.lambda_param * t)
        margins = y_batch * (X_batch.dot(self.w) + self.b)
        mask = margins < 1
        self.w *= (1 - eta * self.lambda_param)
        if np.any(mask):
            X_violate = X_batch[mask]
            y_violate = y_batch[mask]
            self.w += (eta / self.batch_size) * np.dot(y_violate, X_violate.toarray() if hasattr(X_violate, "toarray") else X_violate)
            self.b += (eta / self.batch_size) * np.sum(y_violate)
        norm_w = np.linalg.norm(self.w)
        factor = min(1, (1.0 / np.sqrt(self.lambda_param)) / (norm_w))
        self.w *= factor
    decision = X.dot(self.w) + self.b
    hinge_losses = np.maximum(0, 1 - y * decision)
    objective = 0.5 * self.lambda_param * np.dot(self.w, self.w) + np.mean(hinge_losses)
    if ep % 10 == 0:
        print(f"Epoch {ep}, Giá trị hàm mục tiêu: {objective:.4f}")
    if abs(previous_objective - objective) < self.tol:
        print(f"Dừng sớm tại epoch {ep}, giá trị hàm mục tiêu thay đổi: {abs(previous_objective - objective):.6f}")
        break
    previous_objective = objective
```

Hình 3.11: Cài đặt SVM với thuật toán Pegasos

Lớp SVM được khởi tạo với các siêu tham số gồm tham số $\lambda_{\text{param}} = 1\text{e-}4$, số epoch $\text{epoch} = 1000$, kích thước batch $\text{batch_size} = 256$, ngưỡng dừng sớm $\text{tol} = 1\text{e-}4$ và $\text{random_state} = 42$.

Thuật toán Pegasos giải bài toán tối thiểu hóa hàm mục tiêu dạng:

$$f(w) = \frac{\lambda}{2} \|w\|^2 + \frac{1}{|A_t|} \sum_{i \in A_t} \max(0, 1 - y_i \cdot (w \cdot x_i))$$

Trong đó:

- $\frac{\lambda}{2} \|w\|^2$: Là số hạng chính quy hoá kiểm soát độ phức tạp của mô hình
- $\frac{1}{|A_t|} \sum_{i \in A_t} \max(0, 1 - y_i \cdot (w \cdot x_i))$: Là hàm mất mát trung bình tính trên mini batch A_t .

Quy trình huấn luyện trong phương thức $fit(X, y)$ diễn ra như sau: Nhãn được chuyển từ $\{0, 1\}$ sang $\{-1, +1\} \rightarrow$ Vector trọng số w được khởi tạo $= 0$, với kiểu dữ liệu float32 để tối ưu bộ nhớ, và $b = 0 \rightarrow$ Biến đếm bước cập nhật t được khởi tạo bằng 0.

Trong mỗi chu kì hoàn chỉnh qua toàn bộ tập dữ liệu (mỗi epoch), các chỉ số mẫu được xáo trộn ngẫu nhiên và chia thành các mini-batch có kích thước $batch_size$. Với mỗi mini-batch, bước cập nhật được thực hiện theo công thức Pegasos:

- Bước 1- Tính learning rate thích nghi: $\eta = \frac{1}{\lambda t}$. t là bước cập nhật hiện tại.
Learning rate giảm dần theo thời gian giúp thuật toán hội tụ ổn định
- Bước 2 - Tính margin và xác định mẫu vi phạm: Với mỗi mẫu trong batch, margin được tính là $y_i \cdot (w \cdot x_i + b)$. Các mẫu có margin nhỏ hơn 1 được gọi là mẫu vi phạm.
- Bước 3 - Cập nhật weight decay: $w \leftarrow (1 - \eta \cdot \lambda) \cdot w$, thực hiện giảm trọng số để kiểm soát tham số chuẩn hoá.
- Bước 4 - Cập nhật gradient từ hinge loss: Nếu có tồn tại vi phạm, cập nhật $w \leftarrow w + \frac{\eta}{|A_t|} \sum_{i:vi\ phạm} y_i \cdot x_i$ và cập nhật bias $b \leftarrow b + \frac{\eta}{|A_t|} \sum_{i:vi\ phạm} y_i$
- Bước 5 - Projection: Để đảm bảo nhiệm nằm trong miền khả thi, thực hiện chiếu vector trọng số về cầu hình có bán kính $\frac{1}{\sqrt{\lambda}}$ theo công thức $w \leftarrow \min(1, \frac{1}{\sqrt{\lambda}}) \cdot w$

Đối với việc dự đoán, hàm $predict(X)$ tính điểm quyết định $d = X \cdot w + b$ và trả về nhãn 1 (email là spam) nếu $d \geq 0$ và trả về 0 nếu ngược lại.

Sau khi huấn luyện xong trên toàn bộ 83.448 mẫu của bộ dữ liệu gốc (66.758 mẫu sau khi loại các giá trị thiếu trong quá trình tiền xử lý), mô hình và vectorizer được lưu thành hai file *linear_svm.pkl* và *vectorizer.pkl* sử dụng thư viện pickle, phục vụ cho giai đoạn triển khai ứng dụng.

3.6 Huấn luyện mô hình

3.6.1 Quy trình huấn luyện

Quy trình huấn luyện được thực hiện trong các notebook Python, bao gồm các bước: tải tập mẫu, áp dụng toàn bộ pipeline tiền xử lý, vector hóa bằng TF-IDF, chia train/test theo tỉ lệ 80/20, sau đó gọi phương thức fit của mô hình tương ứng trên tập huấn luyện.

Để đánh giá hiệu năng của các phương pháp theo quy mô dữ liệu, thực nghiệm được lặp lại với các kích thước tập mẫu khác nhau từ 1.000 đến hơn 80.000 mẫu. Thời gian huấn luyện và các chỉ số đánh giá được ghi lại vào các file CSV trong thư mục result/ để phân tích và so sánh.

3.6.2 Tiêu chí đánh giá

Mô hình được đánh giá thông qua bốn chỉ số tiêu chuẩn trong bài toán phân loại nhị phân, được tính toán dựa trên ma trận nhầm lẫn (confusion matrix) với bốn thành phần TP, FP, TN, FN.

- Accuracy phản ánh tỷ lệ dự đoán đúng trên toàn bộ tập kiểm tra theo công thức:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- Precision đo lường mức độ tin cậy khi mô hình dự đoán một email là spam:

$$Precision = \frac{TP}{TP + FP}$$

- Recall đo lường khả năng phát hiện đầy đủ các email spam trong tập kiểm tra:

$$Recall = \frac{TP}{TP + FN}$$

- F1-score là trung bình điều hoà giữa Precision và Recall, là chỉ số tổng hợp phản ánh sự cân bằng giữa hai tiêu chí trên, đặc biệt quan trọng khi cần đánh giá hiệu năng toàn diện của hệ thống:

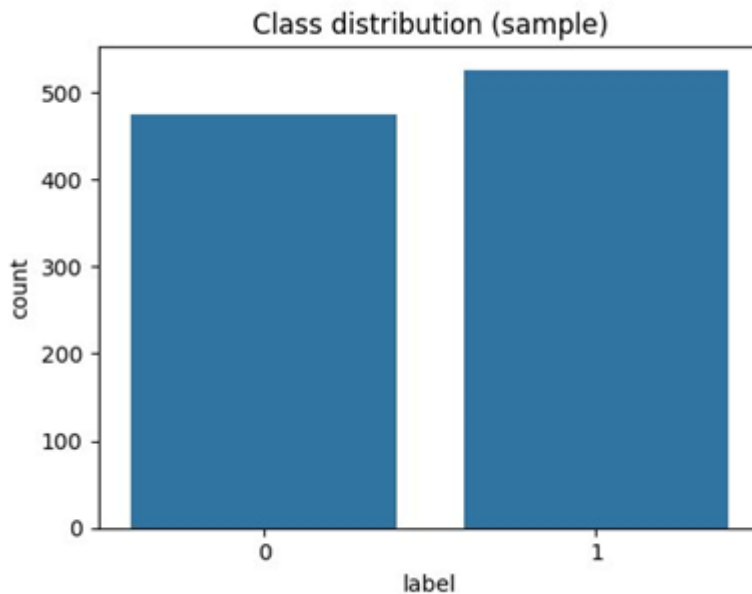
$$F1 = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

CHƯƠNG 4: KẾT QUẢ THỰC NGHIỆM

4.1. Tổng quan thực nghiệm

Nhóm tiến hành thực nghiệm đánh giá ba biến thể mô hình SVM trong bài toán phân loại email rác (spam detection), bao gồm: SVC (sklearn), Hard Margin SVM kết hợp TF-IDF (cài đặt thủ công) và SVM Pegasos (thuật toán tối ưu hóa ngẫu nhiên). Toàn bộ thực nghiệm được thực hiện trên bộ dữ liệu Enron Email Dataset sau khi đã tiền xử lý, với kích thước tập huấn luyện được tăng dần từ 1.000 đến 9.500 mẫu nhằm khảo sát ảnh hưởng của dữ liệu đến hiệu năng mô hình.

Dữ liệu được chia theo tỉ lệ 80/20 (train/test), với phân phối nhãn khá cân bằng giữa hai lớp ham (label 0) và spam (label 1). Đặc trưng được trích xuất bằng phương pháp TF-IDF (Term Frequency – Inverse Document Frequency), kết hợp tiền xử lý văn bản gồm chuyển chữ thường, loại bỏ stopwords và stemming.



Hình 4.1: Phân phối nhãn trong tập dữ liệu mẫu (ham: label 0, spam: label 1)

4.2. Kết quả so sánh các mô hình

Bảng 4.1 trình bày kết quả đánh giá của ba mô hình tại kích thước huấn luyện lớn nhất khả dụng cho mỗi mô hình. Đối với SVC và SVM Pegasos, kết quả được lấy tại $n = 9.500$ mẫu; đối với Hard Margin SVM, do thời gian huấn luyện tăng đột biến theo bậc hai, thực nghiệm chỉ khả thi đến $n = 5.000$ mẫu.

Bảng 4.1: Kết quả đánh giá tổng thể các mô hình SVM

Mô hình	Accuracy	F1-Score	Thời gian (s)
SVC (sklearn)	97,95%	98,05%	600,29
SVM Pegasos	98,16%	98,27%	18,87
Hard Margin SVM + TF-IDF	96,90%	97,08%	10635,54

Kết quả cho thấy SVM Pegasos đạt hiệu năng phân loại tốt nhất với accuracy 98,16% và F1-score 98,27%, đồng thời có thời gian huấn luyện nhanh nhất chỉ khoảng 18,87 giây. SVC (sklearn) đạt kết quả sát với Pegasos (accuracy 97,95%, F1-score 98,05%) nhưng thời gian huấn luyện dài hơn đáng kể (600,29 giây). Hard Margin SVM có hiệu năng thấp nhất trong ba mô hình và tốn thời gian huấn luyện lớn nhất, lên tới hơn 10.635 giây tại $n = 5.000$ mẫu.

4.3. Ảnh hưởng của kích thước tập huấn luyện

4.3.1. Độ chính xác và F1-Score

Bảng 4.2 thống kê kết quả accuracy và F1-score của SVC và SVM Pegasos theo từng mức kích thước dữ liệu huấn luyện. Hard Margin SVM không được đưa vào bảng so sánh này do giới hạn ở $n = 5.000$ mẫu.

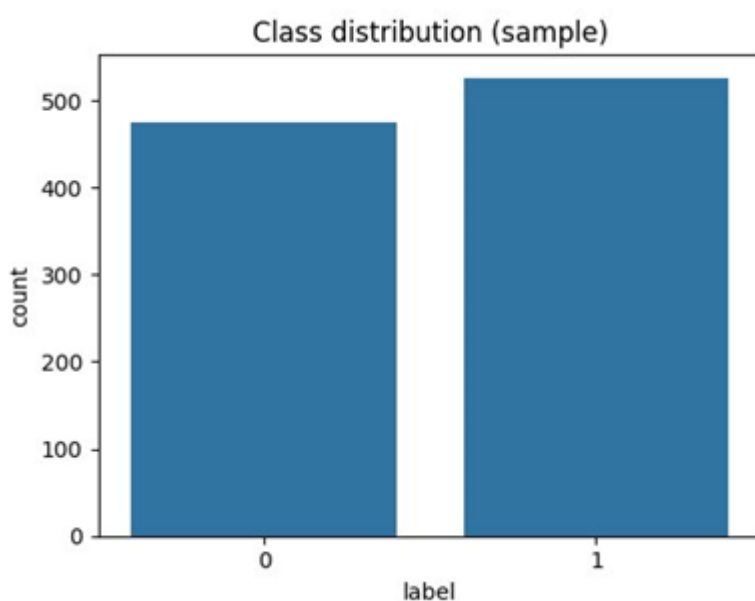
Bảng 4.2: Kết quả SVC và SVM Pegasos theo kích thước huấn luyện

n	SVC Acc	SVC F1	SVC Time	Pegasos Acc	Pegasos F1	Pegasos Time
1.000	94,00%	94,29%	7,92	95,00%	94,95%	1,57
3.000	96,33%	96,39%	62,30	96,83%	97,16%	4,92
5.000	96,90%	97,11%	172,49	97,40%	97,53%	10,51
7.000	97,71%	97,85%	324,92	98,14%	98,28%	13,56
9.500	97,95%	98,05%	600,29	98,16%	98,27%	18,87

Nhìn chung, accuracy và F1-score của cả hai mô hình đều có xu hướng tăng khi kích thước tập huấn luyện tăng. Tuy nhiên, mức độ cải thiện không đồng đều - giai đoạn

từ $n = 1.000$ đến $n = 3.000$ ghi nhận mức tăng rõ rệt nhất, trong khi từ $n = 5.000$ trở đi, hiệu năng ổn định dần và tiến gần về ngưỡng 98%.

Đáng chú ý là tại $n = 1.500$, SVM Pegasos có sự sụt giảm bất thường về F1-score xuống 91,17%. Hiện tượng này có thể được giải thích bởi tính ngẫu nhiên trong quá trình lấy mẫu dữ liệu và đặc thù của thuật toán tối ưu hóa ngẫu nhiên (SGD-based), vốn nhạy cảm hơn với phân phối dữ liệu trong batch nhỏ. Tuy nhiên, mô hình nhanh chóng hồi phục ở các mức n tiếp theo.



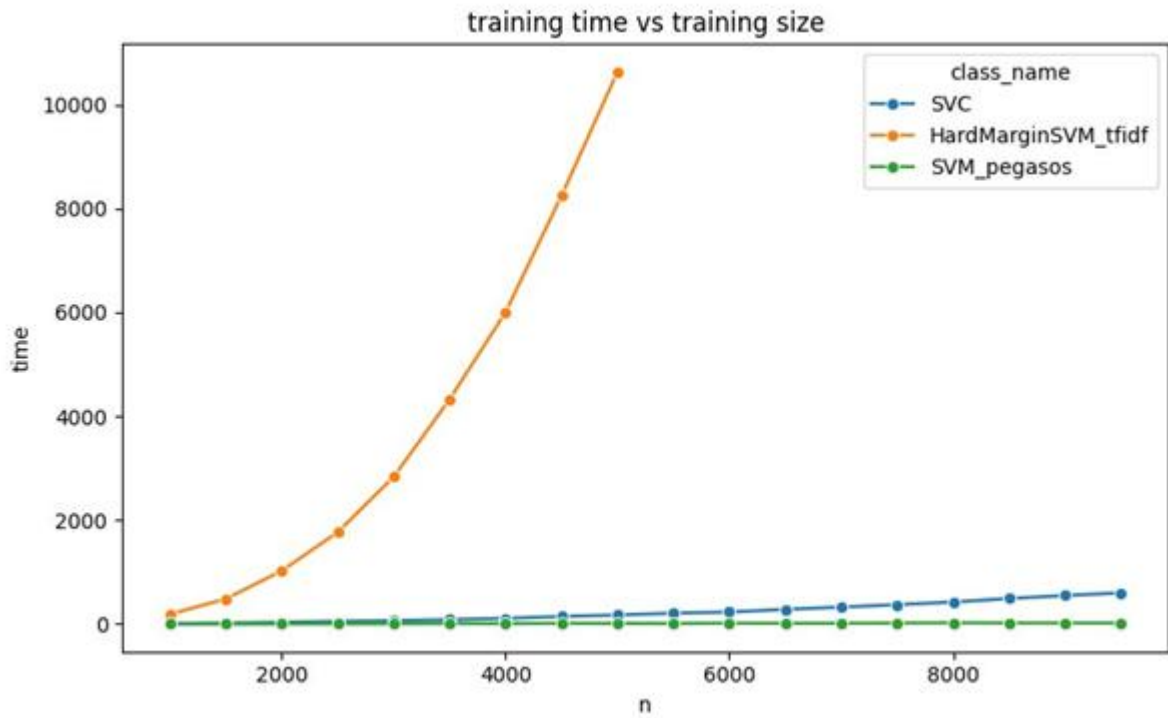
Hình 4.2: Accuracy của ba mô hình theo kích thước tập huấn luyện



Hình 4.3: F1-Score của ba mô hình theo kích thước tập huấn luyện

4.3.2. Thời gian huấn luyện

Hình 4.4 minh họa rõ ràng sự khác biệt về độ phức tạp thời gian huấn luyện giữa ba mô hình. Hard Margin SVM có độ phức tạp tính toán $O(n^3)$ đặc trưng của bài toán tối ưu bậc hai (QP), khiến thời gian tăng trưởng theo cấp số nhân và chỉ có thể chạy đến $n = 5.000$ mẫu (~10.635 giây). SVC của sklearn sử dụng thuật toán SMO (Sequential Minimal Optimization) cho phép mở rộng đến $n = 9.500$ mẫu với thời gian tăng tuyến tính đến bậc hai (600 giây). SVM Pegasos, với thuật toán Stochastic Gradient Descent, có thời gian huấn luyện gần như tuyến tính $O(n)$, chỉ mất khoảng 19 giây cho toàn bộ 9.500 mẫu.



Hình 4.4: Thời gian huấn luyện (ms) của ba mô hình theo kích thước tập huấn luyện

Kết quả thời gian huấn luyện cho thấy SVM Pegasos vượt trội hoàn toàn về tốc độ, nhanh hơn SVC khoảng 32 lần và nhanh hơn Hard Margin SVM hàng trăm lần tại cùng kích thước dữ liệu. Điều này khẳng định lợi thế của Pegasos trong các ứng dụng yêu cầu khả năng mở rộng (scalability) với dữ liệu lớn.

4.4. So sánh với mô hình học máy khác

Bảng 4.3: Đánh giá so sánh SVM và các mô hình khác

STT	Size	Model	Thời gian	Accuracy	F1-score
1	7600	SVM (Scikit-learn)	19.333504	0.983158	0.984298
2	7600	Random Forest	29.179191	0.974737	0.976331
3	7600	Logistic Regression	0.669001	0.977368	0.979014

4.5. Kết quả triển khai hệ thống

Sau khi hoàn thành quá trình huấn luyện trên toàn bộ 83.448 mẫu của bộ dữ liệu gốc (66.758 mẫu sau khi loại bỏ các giá trị thiếu trong quá trình tiền xử lý), mô hình SVC tuyến tính và vectorizer TF-IDF được lưu thành hai file nhị phân `linear_svm.pkl` và `vectorizer.pkl` sử dụng thư viện `pickle`, phục vụ cho giai đoạn triển khai ứng dụng.

Hệ thống được triển khai dưới dạng ứng dụng web sử dụng framework `Flask`, đóng gói bằng `Docker` để đảm bảo tính nhất quán môi trường thực thi. Giao diện cho phép người dùng nhập nội dung email bất kỳ, hệ thống sẽ thực hiện tiền xử lý (chuyển chữ thường, loại bỏ stopwords, stemming), chuyển đổi thành vector TF-IDF và đưa qua mô hình để dự đoán nhãn spam hoặc ham. Ngoài kết quả phân loại, giao diện còn hiển thị văn bản sau khi tiền xử lý, giúp người dùng kiểm chứng quá trình xử lý của hệ thống.

Deploy 

Demo phân loại email tiếng Anh spam

Nhập nội dung email:

Subject: Project Schedule and Documentation Update
Content: Hi Team, I have attached the updated project schedule in the attached xls file.
Please review the doc and let me know if the timelines work for your department.
Thank you for your cooperation.

Best regards,
Vincent

Kiểm tra

☒ Email có khả năng không phải là SPAM

Văn bản sau khi được tiền xử lý:

subject project schedul document updat contenthi teami attach updat project schedul attach xls file pleas
review doc let know timelin work depart thank cooper best regard vincent

Hình 4.7: Giao diện hệ thống phân loại email - kết quả: không phải SPAM (ham)

Hình 4.7 minh họa trường hợp người dùng nhập nội dung email hợp lệ - đoạn văn bản mô tả quá trình lưu mô hình sau huấn luyện. Hệ thống trả về kết quả “Email có khả năng không phải là SPAM” với nền xanh lá, đồng thời hiển thị văn bản đã qua tiền xử lý phía dưới để người dùng có thể kiểm chứng.

Deploy

Demo phân loại email tiếng Anh spam

Nhập nội dung email:

Subject: Special Offer: Invest in Your Health Today!
Content: Dear Customer, We have an exclusive product just for you.
This new med treatment is designed to improve your life quality significantly.
Do not waste your money; invest in our professional account services now.
Visit our site at <http://special-deals.hk> for more details.
To remove your address from our mailing list, please click the stop button below.

Kiểm tra

 Email có khả năng là SPAM

Văn bản sau khi được tiền xử lý:

subject special offer invest your health today contentdear custom we exclus product this new med treatment design improv life qualiti signific do wast money invest profession account servic visit site detail to remov address mail list pleas click stop button

Hình 4.8: Giao diện hệ thống phân loại email - kết quả: có khả năng là SPAM

4.6. Nhận xét và đánh giá

Qua các thực nghiệm, nhóm rút ra một số nhận xét sau:

1. SVM Pegasos là lựa chọn tối ưu nhất trong ba mô hình: đạt độ chính xác cao nhất (98,16%), F1-score cao nhất (98,27%) và thời gian huấn luyện nhanh nhất (~19 giây). Đây là mô hình phù hợp nhất để triển khai trong môi trường thực tế với dữ liệu lớn.

2. SVC (sklearn) cho kết quả phân loại tốt và ổn định, tuy nhiên thời gian huấn luyện tăng đáng kể khi dữ liệu lớn do đặc tính của thuật toán SMO. Mô hình phù hợp cho các bài toán quy mô vừa với yêu cầu ổn định cao.
3. Hard Margin SVM có hiệu năng thấp nhất và thời gian huấn luyện tăng theo hàm bậc ba, không khả thi với dữ liệu lớn hơn 5.000 mẫu. Mô hình này chủ yếu có giá trị nghiên cứu lý thuyết về thuật toán tối ưu QP.
4. Kích thước tập huấn luyện có ảnh hưởng tích cực đến hiệu năng mô hình, nhưng lợi ích biên giảm dần khi $n > 5.000$. Điều này gợi ý rằng với bộ dữ liệu này, khoảng 5.000–7.000 mẫu đã đủ để mô hình đạt hiệu năng gần tối ưu.
5. Phân tích đặc trưng quan trọng cho thấy mô hình học được các từ có ý nghĩa ngữ nghĩa phù hợp với bản chất của email spam và ham, khẳng định tính hợp lý của pipeline tiền xử lý và trích xuất đặc trưng TF-IDF đã áp dụng.

CHƯƠNG 5: KẾT LUẬN

Hệ thống phân loại email rác tự động được xây dựng đã chứng minh rằng các thuật toán học máy, đặc biệt là Support Vector Machine (SVM) tối ưu hóa bằng phương pháp Pegasos SGD, có thể đem lại kết quả phân loại cực kỳ chính xác với độ chính xác cao trên bộ dữ liệu quy mô lớn. Dù còn những hạn chế nhất định về việc xử lý email đa ngôn ngữ ngoài tiếng Anh, kiến trúc hệ thống đã đáp ứng tốt các yêu cầu về chuẩn hóa văn bản thông qua thư viện NLTK và biểu diễn đặc trưng hiệu quả bằng kỹ thuật TF-IDF.

Thông qua quá trình thực hiện, nhóm chúng em đã tích lũy được nhiều kiến thức quý báu về xử lý ngôn ngữ tự nhiên, hiểu rõ bản chất toán học của các biến thể SVM và quy trình thiết kế một hệ thống học máy toàn diện từ khâu thu thập, tiền xử lý dữ liệu đến triển khai ứng dụng web thực tế. Việc tự cài đặt các mô hình như Hard-margin SVM và Pegasos SGD cũng giúp nhóm hiểu rõ tầm quan trọng của việc lựa chọn thuật toán phù hợp để cân bằng giữa thời gian huấn luyện và hiệu năng tính toán trong bài toán dữ liệu lớn.

Cuối cùng, nhóm hy vọng những kết quả nghiên cứu này sẽ là nền tảng vững chắc để phát triển các hệ thống lọc thư rác thông minh và chuyên sâu hơn. Việc hoàn thiện quy trình từ tiền xử lý, trích xuất đặc trưng đến tối ưu hóa mô hình học máy sẽ góp phần xây dựng các ứng dụng bảo mật mạnh mẽ, mang lại giá trị thiết thực trong việc bảo vệ an toàn thông tin và nâng cao năng suất làm việc cho người dùng trong môi trường số hóa hiện nay.

Tài liệu tham khảo

1. Harris Drucker, Donghui Wu, Vladimir N. Vapnik, Support Vector Machines for Spam Categorization. Ed. IEEE Transactions on Neural Networks, Vol. 10, No. 05, 1999.
2. Shai Shalev-Shwartz, Yoram Singer, Nathan Srebro, Andrew Cotter, Pegasos: Primal Estimated sub-GrAdient Solver for SVM. Ed. Mathematical Programming, Vol. 127, No. 01, 2011.
3. Thorsten Joachims, Text Categorization with Support Vector Machines: Learning with Many Relevant Features. Ed. Machine Learning: ECML-98 (Lecture Notes in Computer Science), Vol. 1398, 1998.
4. Gerard Salton, Christopher Buckley, Term-weighting approaches in automatic text retrieval. Ed. Information Processing & Management, Vol. 24, No. 05, 1988.